

The First 30 Days

A Fractional CTO's Field Checklist

Every engagement starts the same way — you walk into a codebase, a team, and a set of problems no one fully agrees on. This is the checklist I run through in the first 30 days. What to diagnose, what to fix immediately, and (just as important) what to leave alone.

Week 1: Diagnose

Goal: Understand the real system — not the one described in the interview.

Do

- ☐ **Map the actual delivery path.** How does work move from idea to ticket to code to production? Talk to 3 engineers individually. You'll get 3 different answers. That's the diagnosis.
- ☐ **Find the deployment bottleneck.** How often does the team ship? What blocks releases? If deploys are manual, painful, or require a specific person — that's your first signal.
- ☐ **Identify the “bus factor” risks.** Who has knowledge that exists nowhere else? Which systems break when someone's on vacation?
- ☐ **Audit the decision-making pattern.** Where do decisions stall? Who has to approve what? Most teams have at least one invisible approval bottleneck.
- ☐ **Get the numbers (if they exist).** Cycle time, deployment frequency, incident rate, time-to-recovery. If they don't track these — that's a finding, not a problem to solve yet.

Leave Alone

- The tech stack. Unless something is on fire, the stack is not the problem in Week 1.
 - Team structure. You don't know enough yet. Reorganizing in Week 1 is malpractice.
 - “Quick” architectural rewrites someone's been pitching for months. Smile. Nod. Keep diagnosing.
-

Week 2: Quick Wins

Goal: Build trust by fixing things the team has been complaining about.

Do

- ☐ **Fix the most painful part of the deploy process.** Almost always the highest-leverage move. Even a 50% improvement here changes team morale.
- ☐ **Remove one unnecessary approval gate.** Find the review step that exists because someone added it during a crisis two years ago. Kill it (with team buy-in).
- ☐ **Set up a 30-min weekly sync with the founder/CEO.** Not a status update — a decision-making session. Bring 2–3 decisions, get them made, move on.
- ☐ **Document one thing that only exists in someone's head.** Pick the scariest bus-factor item from Week 1. Get it written down.

Leave Alone

- Performance reviews or feedback processes. Too early. You haven't earned the standing yet.
- Salary or hiring conversations. Unless there's an active crisis, this is a Week 4+ topic.
- Anything the team is emotionally attached to, even if it's inefficient. Earn trust first.

Week 3: Install Systems

Goal: Create the operating rhythm that will outlast you.

Do

- ☐ **Establish a delivery cadence.** If the team doesn't have a predictable release rhythm, set one. Weekly or biweekly — what matters is consistency.
- ☐ **Introduce a lightweight decision framework.** Who decides? By when? What happens if we don't? Write it on a whiteboard. Within two weeks the team starts using it without you.
- ☐ **Set up incident response (or fix the broken one).** On-call rotation, runbooks for the top 3 failure modes, a blameless post-incident review that actually happens.
- ☐ **Start a weekly metrics review.** 15 minutes. Cycle time, deployment frequency, open incidents. No vanity metrics. Establish a baseline.

Leave Alone

- Rewriting the CI/CD pipeline from scratch. Improve incrementally. A full rebuild is a Q2 project at earliest.
 - Introducing new tools unless they solve a problem the team already feels.
-

Week 4: Baseline + Handoff Plan

Goal: Make progress measurable. Start making yourself replaceable.

Do

- ☐ **Document your baselines.** Cycle time, deploy frequency, incident response time, decision velocity. This is your “before” picture.
- ☐ **Create a 90-day roadmap.** Not a wishlist — 3 priorities, sequenced, with owners. The roadmap should make sense even if you disappeared tomorrow.
- ☐ **Identify what the team can now own without you.** Which systems are self-sustaining? Which still need your involvement? Be honest.
- ☐ **Have the “what does success look like?” conversation.** Client-side KPIs, not activity metrics. Not “we had 12 meetings” — “cycle time dropped 40%.”

Leave Alone

- Trying to “finish” anything. Week 4 is about setting direction, not completion.
- Making yourself indispensable. The goal is the opposite. A fractional CTO who can't be replaced built a dependency, not a system.

The Pattern

Week 1 → listen and measure

Week 2 → earn trust through visible fixes

Week 3 → install the rhythm

Week 4 → prove the baseline and plan the exit

The best fractional CTOs change the operating system, not the org chart. By Day 30, the team should be shipping differently — and they should understand why.